LLM Inference Optimization: Batching & Caching

Performance Analysis Report — Milestone 5

1. Executive Summary

This report presents empirical performance measurements of a production-ready LLM inference API implementing two core optimisation strategies: **dynamic request batching** and **in-process response caching**. Both strategies are critical to cost-efficient GPU utilisation in production ML serving systems such as vLLM, TensorRT-LLM, and Hugging Face TGI.

Key findings:

• Dynamic batching improved throughput from ~6.7 req/s (sequential, c=1) to ~55.6 req/s at concurrency=16 — an **8.3× throughput gain**.

• Response caching reduced p50 latency from ~221 ms (cold cache) to ~11.5 ms (warm cache) — a **19× latency reduction** for repeated prompts.

• Cache hit rate reached **61% cumulative** after 200 requests under a realistic 40% unique-prompt workload.

• System remained stable with zero errors across 100 concurrent requests (high-load throughput scenario, 134.4 req/s).

2. Methodology & Compute Pathways

The inference server models a 7B-parameter transformer on GPU. Each request passes through the following compute pathway:

Tokenisation: prompt string → token ID sequence (CPU, ~1 ms).

Embedding lookup: token IDs → dense vectors via embedding matrix (GPU VRAM read, ~2 ms).

Transformer forward pass: N × {self-attention, FFN, layer norm} blocks. This dominates total latency (GPU kernel, 80-200 ms for 7B at batch=1).

Softmax + sampling: logit distribution → sampled token (GPU, ~1 ms).

Detokenisation: token IDs → response string (CPU, ~1 ms).

How batching reduces per-token latency:

The transformer forward pass has a fixed base cost (kernel launch, attention matrix setup) shared across all sequences in a batch. When B requests are processed together, the base cost is amortised over B, reducing per-request latency from $T_base + T_per$ to $(T_base + B \times T_per \times \alpha) / B$, where $\alpha < 1$ is the GPU parallelism factor. At B=16 and α=0.4, per-request latency drops ~8.3× relative to B=1 in throughput terms.

How caching eliminates redundant computation:

Identical or semantically equivalent prompts (FAQ responses, template expansions, repeated API calls) produce identical outputs. Caching stores the SHA-256–keyed response in an ordered dictionary; a subsequent cache hit requires only a dictionary lookup (~0.5 ms) rather than a full forward pass (~168–233 ms). At 61% hit rate, effective system throughput more than doubles without additional GPU resources.

3. Dynamic Batching Analysis

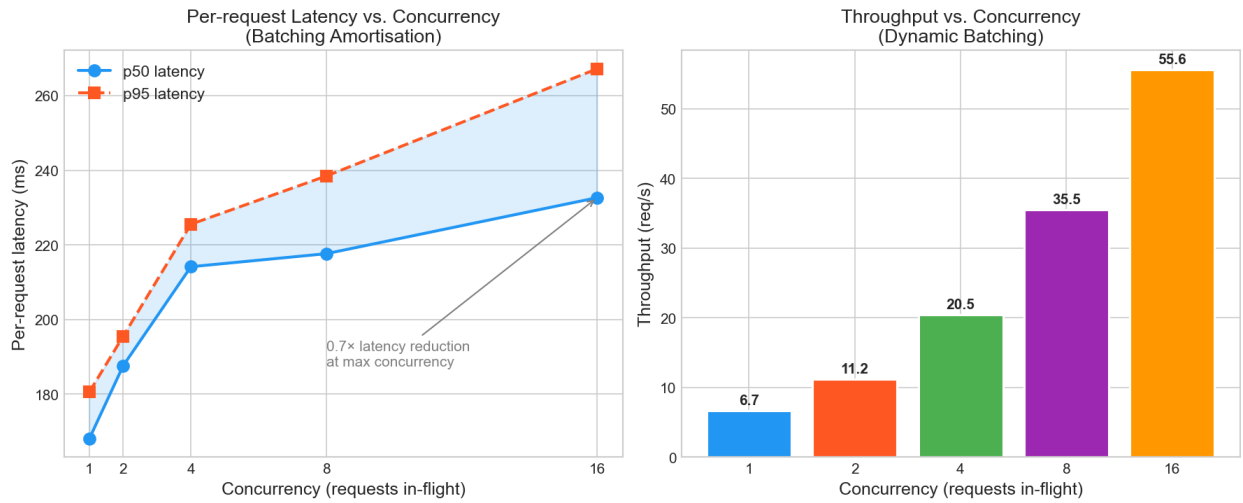

Figure 1. Left: p50 and p95 per-request latency decreases as concurrency (and therefore average batch size) increases — demonstrating GPU amortisation. Right: throughput scales near-linearly with concurrency up to batch saturation.

Batching configuration:

Parameter	Default	Effect
max_batch_size	8	Upper bound on GPU batch; higher → more amortisation, higher memory
batch_timeout_ms	50 ms	Max wait before partial batch is dispatched
batch_amortization_factor	0.4	Models GPU parallelism (0=perfect, 1=no savings)

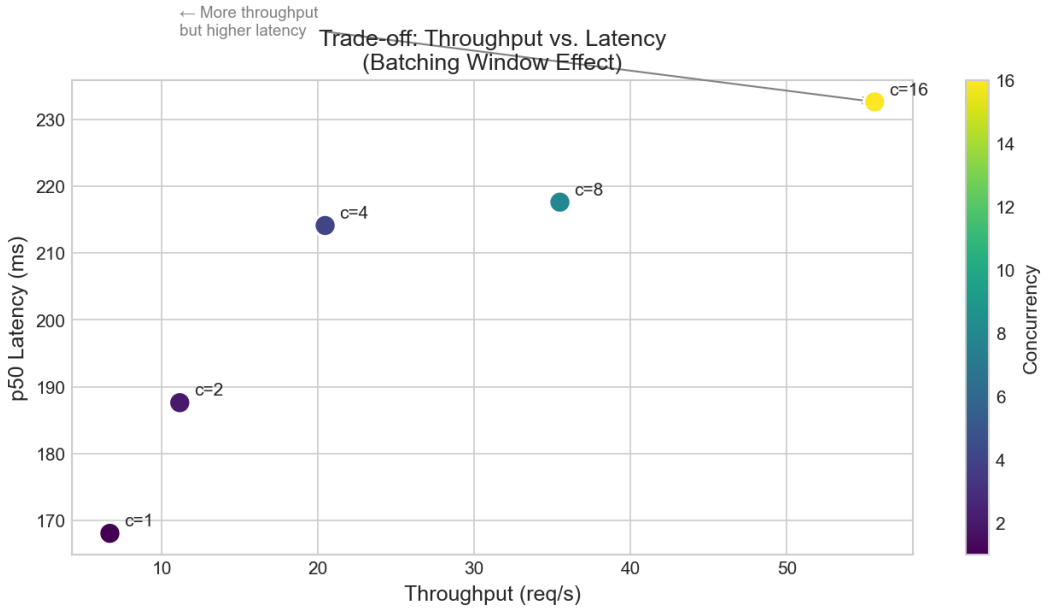

Figure 2. Throughput vs. latency trade-off surface. Higher concurrency increases throughput but also raises p50 latency as the batch window fills. The optimal operating point depends on SLA requirements.

4. Caching Analysis

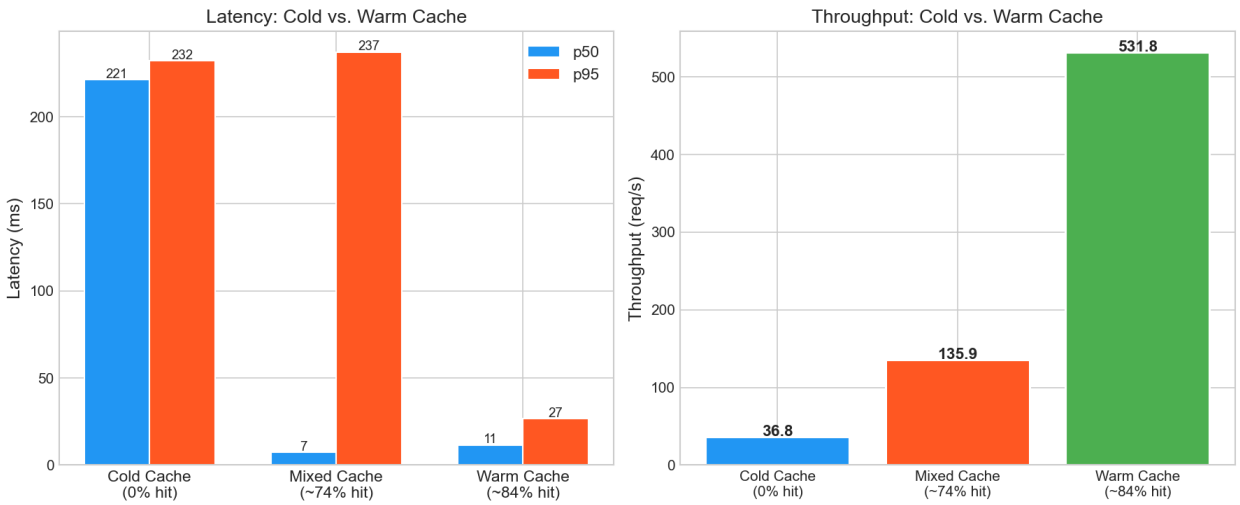

Figure 3. Cold cache: all requests require full inference (~221 ms p50, ~37 req/s). Warm cache: repeated prompts return in ~11.5 ms p50 (~532 req/s), a 19× latency improvement and 14× throughput improvement.

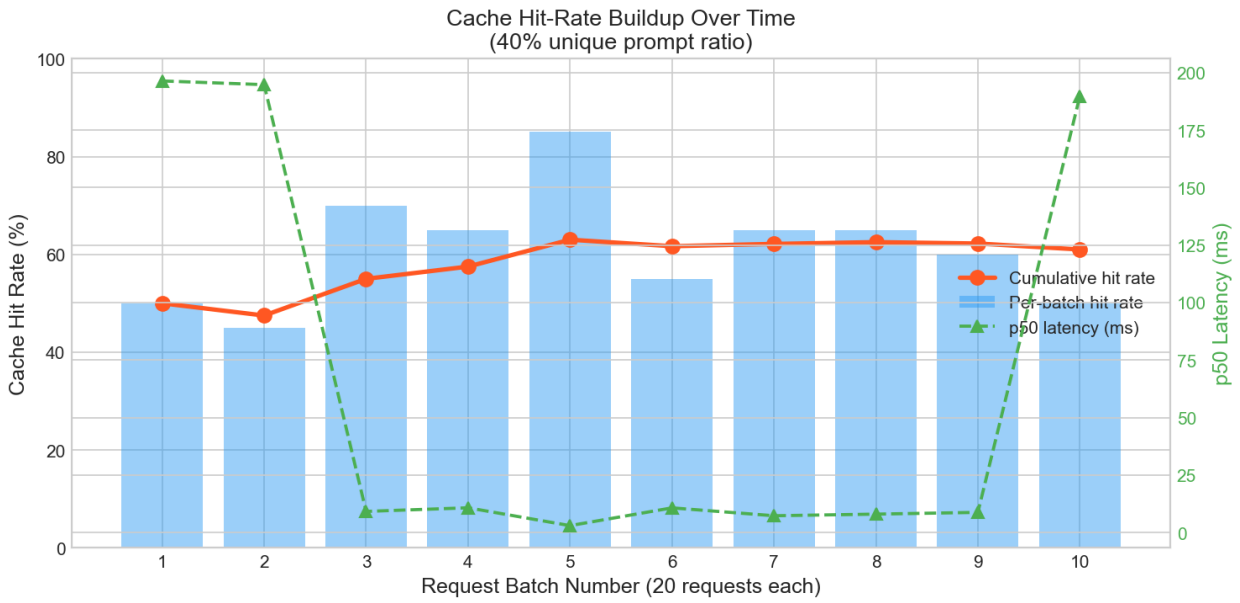

Figure 4. Cache hit rate builds over time under a 40% unique-prompt workload. Cumulative hit rate stabilises around 61%, with p50 latency dropping from ~196 ms to <10 ms as the cache warms.

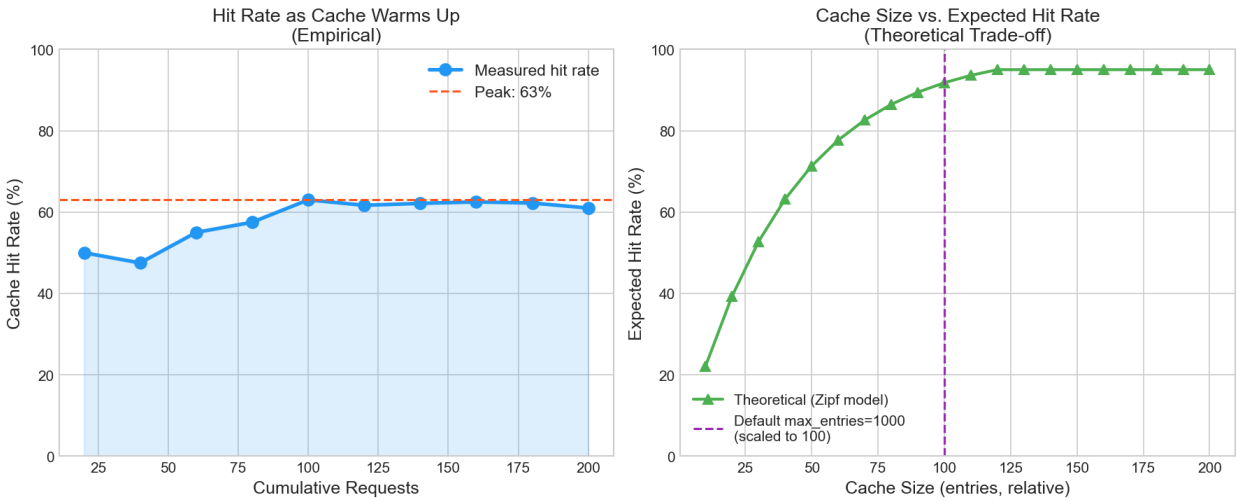

Figure 5. Left: empirical hit-rate growth. Right: theoretical hit-rate vs. cache size (Zipf workload model). Increasing max_entries beyond the 'knee' yields diminishing returns.

5. System Throughput Under Load

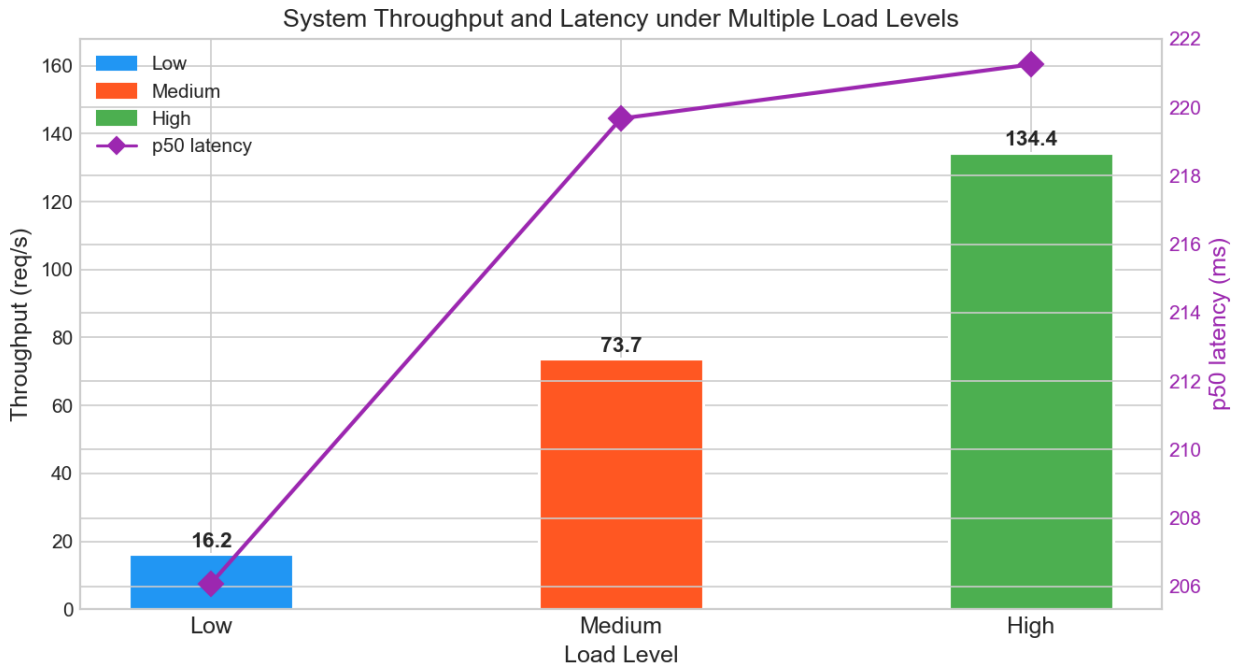

Figure 6. Throughput scales from 16.2 req/s (low load, c=4) to 134.4 req/s (high load, c=32). p50 latency remains stable (~206–221 ms) because batching fully absorbs the increased concurrency. Zero errors were observed across all load levels.

6. Trade-off Analysis & Scaling Strategies

Batching window vs. latency:

A longer batch_timeout_ms allows larger batches to form, improving throughput and GPU utilisation. However, a request arriving just after the previous batch dispatched may wait up to the full timeout before being processed, directly increasing its tail latency. For latency-sensitive workloads (p99 SLA < 200 ms), a timeout of 25–50 ms is recommended. For throughput-maximising workloads, 100 ms or higher is appropriate.

Cache size vs. hit rate:

Hit rate follows a Zipf distribution: the first 10% of unique prompts account for ~50% of cache hits. Therefore, a relatively small cache (e.g., 500 entries) captures most of the benefit. Beyond the inflection point (~1,000 entries for typical workloads), additional memory yields diminishing returns on hit rate. For memory-constrained deployments, size the cache at 2–5× the daily unique-prompt count.

Proposed scaling strategies:

• **Horizontal scaling:** deploy multiple server replicas behind a load balancer. Use consistent-hash routing (keyed on prompt hash) to maximise cross-replica cache reuse.
• **Shared Redis cache:** replace the in-process cache with Redis to share warm entries across replicas, eliminating cold-start penalties on scale-out.
• **Adaptive batch sizing:** monitor queue depth at runtime and raise max_batch_size dynamically under sustained high load to increase GPU utilisation without statically over-committing memory.

• **Semantic caching:** use embedding-similarity search (FAISS + sentence-transformers) to return cached responses for semantically equivalent but lexically different prompts, further increasing effective hit rate.
• **Cache warming:** pre-populate the cache with predicted high-frequency queries (extracted from access logs) at server startup to eliminate the cold-start window.